

PrimaryButton.js — Fundamentos

Explicación detallada del botón reutilizable (GymBro App)

```
import { StyleSheet, Text, TouchableOpacity } from 'react-native';

export default function PrimaryButton({ title, onPress }) {
  return (
    <TouchableOpacity style={styles.button} onPress={onPress}>
      <Text style={styles.text}>{title}</Text>
    </TouchableOpacity>
  );
}
```

1. Qué es y por qué existe

Es el componente **más simple** de todos los que vimos hasta ahora: un botón amarillo reutilizable para toda la app (lo usan tanto login.js como signup.js, y probablemente otras pantallas). Sin este componente, cada pantalla tendría que repetir el TouchableOpacity + Text + los mismos estilos una y otra vez. Es el ejemplo más puro de **componente de presentación** (“dumb component” / “presentational component”): no tiene estado propio, no tiene lógica, solo recibe props y renderiza UI.

2. Las props (línea 3)

```
export default function PrimaryButton({ title, onPress }) {
```

Usa **destructuring** directo en los parámetros de la función — en vez de recibir un objeto props y después escribir props.title, props.onPress, lo desestructura ahí mismo. Solo tiene dos props, y las dos son obligatorias (no tienen valor por defecto):

- **title**: el texto que se muestra dentro del botón (string). Ej: "Iniciar Sesión", "Registrarme".
- **onPress**: la función que se ejecuta cuando el usuario toca el botón. El componente **no sabe qué hace** esa función — solo la recibe y se la pasa tal cual a TouchableOpacity. Esto es **inversión de control**: la lógica de qué pasa al tocar el botón vive en el componente padre (por ejemplo handleLogin en login.js), no acá.

3. TouchableOpacity (líneas 5-7)

```
<TouchableOpacity style={styles.button} onPress={onPress}>
  <Text style={styles.text}>{title}</Text>
</TouchableOpacity>
```

TouchableOpacity es el componente táctil “de toda la vida” de React Native: envuelve a sus hijos y responde a toques, atenuando la opacidad brevemente al presionar (de ahí el nombre) como feedback visual de que el toque se registró. No tiene lógica interna de estado — simplemente:

- Le pasa el estilo (styles.button) que define el fondo amarillo, tamaño y bordes redondeados.
- Conecta el evento de toque (onPress) directo con la prop que le llegó del padre.
- Adentro, un Text simple muestra el title recibido.

No usa PrimaryButton ningún useState ni ningún hook — es tan simple que ni siquiera necesita ser mencionado como “con estado” o “sin estado”, directamente **no tiene ciclo de vida propio** más allá de renderizar lo que le dan.

4. Los estilos (líneas 11-26)

```
button: {
  width: '100%',
  height: 52,
  backgroundColor: '#FFC107',
  borderRadius: 7,
  alignItems: 'center',
  justifyContent: 'center',
  marginTop: 10,
},
text: {
  color: '#111111',
  fontSize: 17,
  fontWeight: '700',
},
```

- `backgroundColor: '#FFC107'` es el amarillo característico de la app (el mismo color que se usa en `login.js` para el ícono y el link “Regístrate” — es el color de acento de toda la identidad visual de GymBro).
- `width: '100%'` hace que el botón ocupe todo el ancho disponible del contenedor padre (por eso en `login.js` y `signup.js` se ve del mismo ancho que los inputs, sin tener que fijar un `width` numérico).
- `height: 52` coincide exactamente con la altura de `CustomInput` (también 52) — consistencia visual entre inputs y botón.
- `alignItems: 'center' + justifyContent: 'center'` centran el texto tanto horizontal como verticalmente dentro del botón.
- `color: '#111111'` (casi negro) sobre fondo amarillo da buen contraste de legibilidad (accesibilidad básica de color).

5. Cómo se usa

```
// login.js
<PrimaryButton title="Iniciar Sesión" onPress={handleLogin} />
```

```
// signup.js
<PrimaryButton title="Registrarme" onPress={handleRegister} />
```

En ambos casos, el padre define **qué texto mostrar** y **qué función ejecutar**, y `PrimaryButton` solo se encarga de la parte visual + táctil. Si mañana quisieras usarlo en otra pantalla (por ejemplo un botón “Guardar rutina”), simplemente lo importás y le pasás otro `title/onPress` — cero cambios necesarios en `PrimaryButton.js`.

Resumen para el oral

“PrimaryButton es un componente de presentación puro: no tiene estado ni lógica propia, solo recibe `title` y `onPress` como props y renderiza un `TouchableOpacity` estilizado. Es reutilizable en toda la app porque delega completamente la lógica de qué hacer al tocar el botón al componente padre — principio de inversión de control, típico de componentes de UI reutilizables en React.”